2026-05-20 - Writing a Basic Zephyr Application

Goal

Build a simple Zephyr application from scratch and understand the entry point and runtime model.

What you will learn

- How `main()` works in Zephyr
- Using `printk()` and `LOG_*` for output
- Basic thread usage and delays
- Initializing devices at runtime

Overview

A Zephyr app typically starts in `main()`, which runs as a thread under the Zephyr kernel. From there, you can initialize peripherals, configure timers, and start application logic.

Typical code

- `void main(void)` is the app entry point
- `printk()` writes to the system console
- `k_sleep(K_SECONDS(x))` pauses the current thread
- `device_get_binding()` or `DEVICE_DT_GET` obtains peripheral handles

Why this matters

This is the core of application development: writing code that interacts with the kernel and hardware.

Practice tasks

1. Create `src/main.c` with a blinking LED or periodic console message.
2. Add a thread using `K_THREAD_DEFINE` or `k_thread_create`.
3. Use `k_sleep()` and `printk()` to observe behavior.

Next topic

Tomorrow, we will learn how to use Zephyr drivers and device binding properly.